ELO328 - Procesamiento Digital de Imágenes

Clase 17: Contornos

Prof. Nicolás Torres
nicolas.torresr@usm.cl
Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

Representación de objetos

- Típicamente, después de segmentar una imagen viene una etapa llamada *Feature Extraction* (extracción de características), que consiste en detectar características y describir esas características.
- Detectar características permite representar un objeto asignando atributos cuantitativos a las características detectadas.
- Por ejemplo, podríamos detectar esquinas en el borde de una región y describir esas esquinas por su orientación y ubicación, los cuales son atributos cuantitativos.

Representación de Contornos

Representación de contorno es variada:

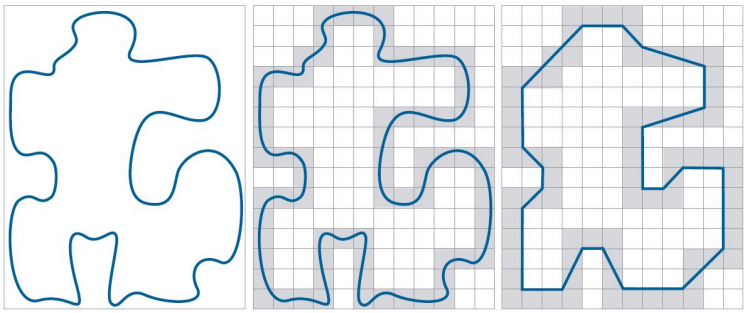
- Los píxeles del borde.
- Puntos representativos.
- Puntos representativos parametrizados (e.g. splines (función definida por partes mediante polinomios)).

Existen métodos que consideran un conjunto fijo de puntos y otros son más flexibles.

La dinámica de seguimiento de contornos también es variada (a estudiar más adelante).

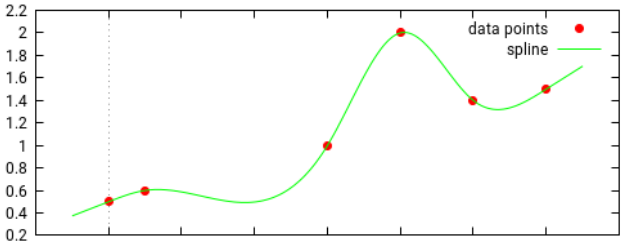
Aproximaciones Poligonales

- El contorno de una imagen se puede aproximar mediante un polígono.
- Se aproxima mediante un conjunto de segmentos de línea recta.



Spline

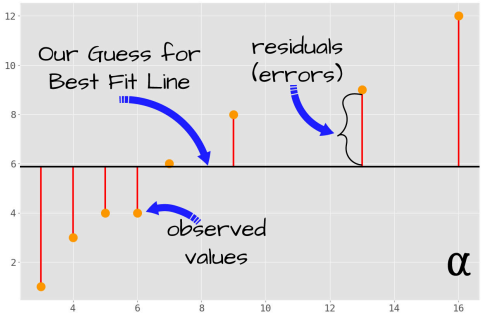
En análisis numérico, un *spline* es una curva diferenciable definida mediante polinomios.

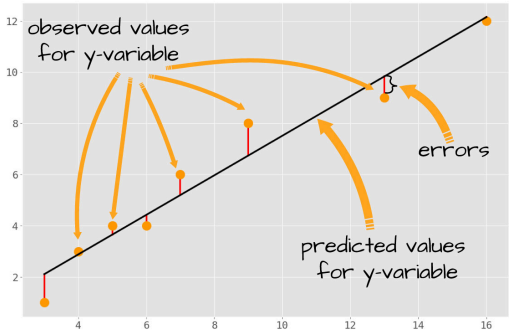


Para encontrar estas curvas se utiliza el método de los **mínimos cuadrados**.

Mínimos Cuadrados

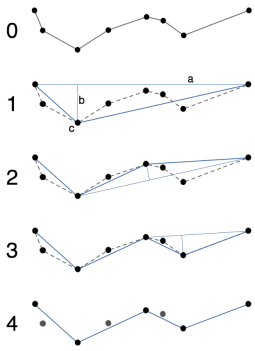
La técnica de mínimos cuadrados es un método estadístico usado para encontrar la curva que mejor se ajuste a los datos minimizando el error.





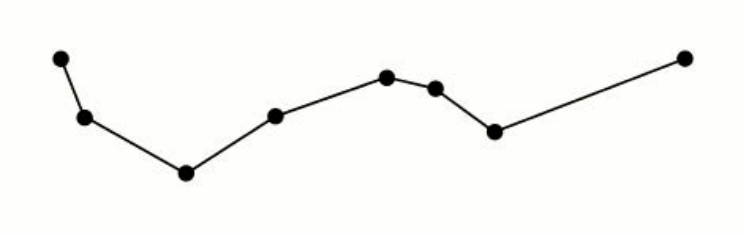
Ramer–Douglas–Peucker

El algoritmo de Douglas-Peucker sirve para reducir el número de puntos utilizados en la aproximación de una curva.

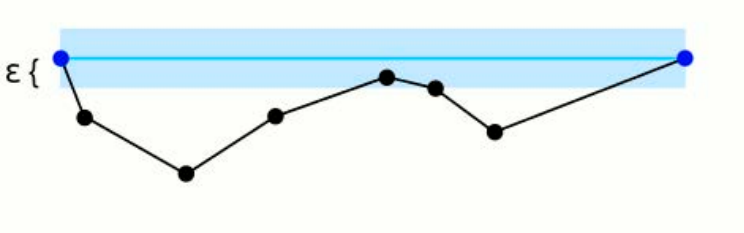


Algoritmo Ramer–Douglas–Peucker • Paso a Paso

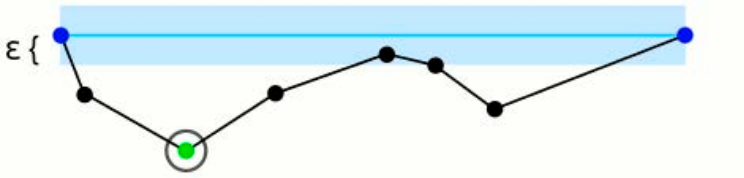
La curva inicial es una lista ordenada de puntos o segmentos y un umbral de error ε > 0. El algoritmo construye una aproximación de la curva inicial mediante un proceso recursivo.



Se toma como solución inicial el segmento que une los dos puntos extremos de la curva.



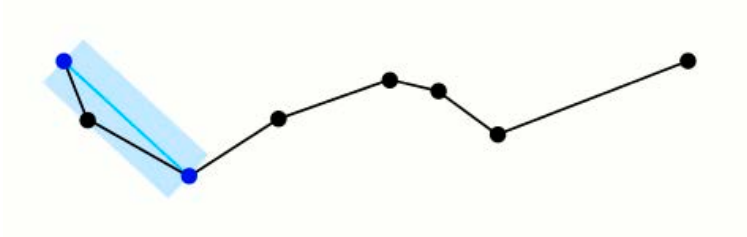
Entonces, se busca el punto a mayor distancia de dicho segmento (peor punto).



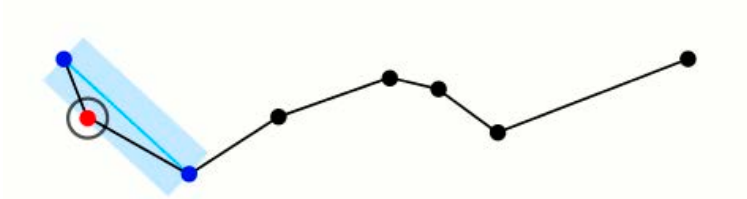
Si la distancia del peor punto al segmento es menor que ε, entonces se termina el proceso. X

Si la distancia del peor punto al segmento es mayor o igual que ε, entonces se repite el proceso anterior de forma recursiva considerando:

1. El contorno de puntos entre el primer y el peor punto.



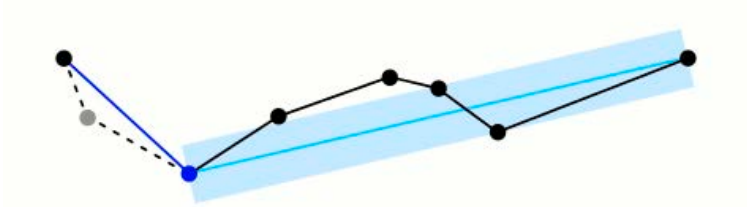
Entonces, se busca el punto a mayor distancia de dicho segmento (peor punto).



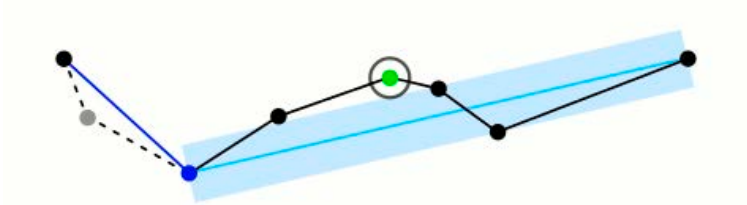
Si la distancia del peor punto al segmento es menor que ε, entonces se termina el proceso. ✓

Si la distancia del peor punto al segmento es mayor o igual que ε, entonces se repite el proceso anterior de forma recursiva considerando:

1. El contorno de puntos entre el primer y el peor punto. ✓
2. El contorno de puntos entre el peor punto y el punto final de la curva.



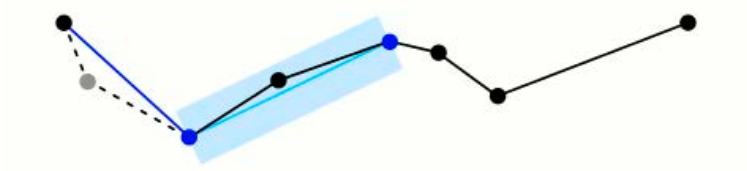
Entonces, se busca el punto a mayor distancia de dicho segmento (peor punto).



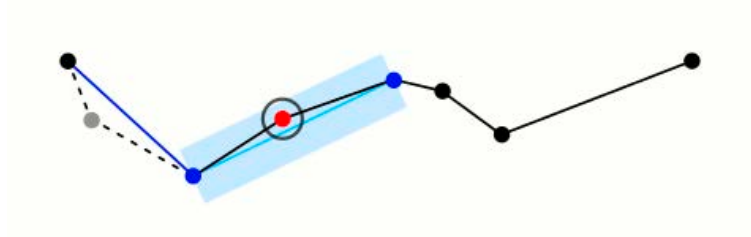
Si la distancia del peor punto al segmento es menor que ε, entonces se termina el proceso. X

Si la distancia del peor punto al segmento es mayor o igual que ε, entonces se repite el proceso anterior de forma recursiva considerando:

1. El contorno de puntos entre el primer y el peor punto.



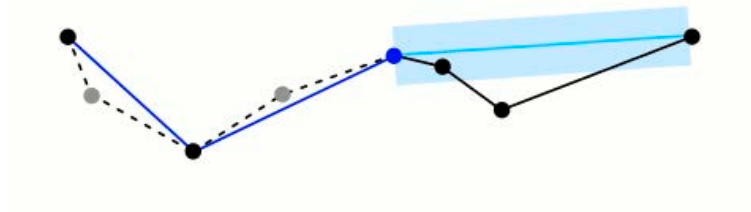
Entonces, se busca el punto a mayor distancia de dicho segmento (peor punto).



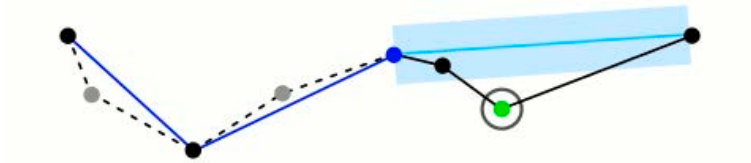
Si la distancia del peor punto al segmento es menor que ε, entonces se termina el proceso. ✓

Si la distancia del peor punto al segmento es mayor o igual que ε, entonces se repite el proceso anterior de forma recursiva considerando:

1. El contorno de puntos entre el primer y el peor punto. ✓
2. El contorno de puntos entre el peor punto y el punto final de la curva.



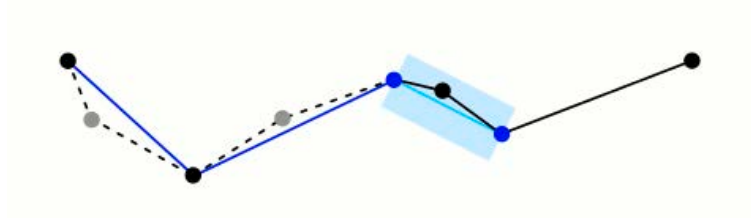
Entonces, se busca el punto a mayor distancia de dicho segmento (peor punto).



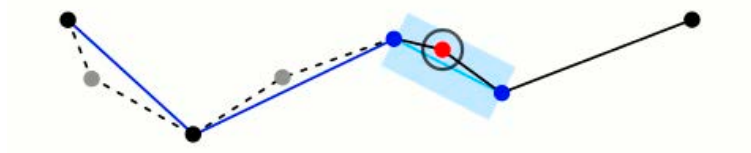
Si la distancia del peor punto al segmento es menor que ε, entonces se termina el proceso. X

Si la distancia del peor punto al segmento es mayor o igual que ε, entonces se repite el proceso anterior de forma recursiva considerando:

1. El contorno de puntos entre el primer y el peor punto.



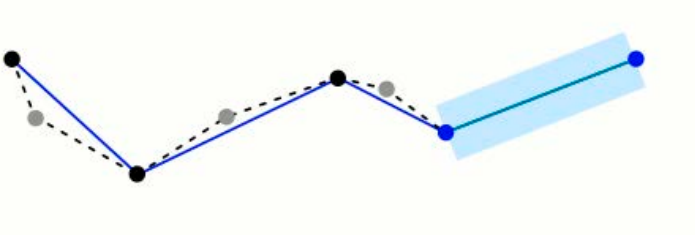
Entonces, se busca el punto a mayor distancia de dicho segmento (peor punto).



Si la distancia del peor punto al segmento es menor que ε, entonces se termina el proceso. ✓

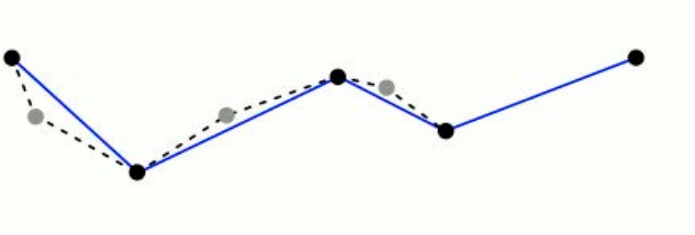
Si la distancia del peor punto al segmento es mayor o igual que ε, entonces se repite el proceso anterior de forma recursiva considerando:

1. El contorno de puntos entre el primer y el peor punto. ✓
2. El contorno de puntos entre el peor punto y el punto final de la curva.

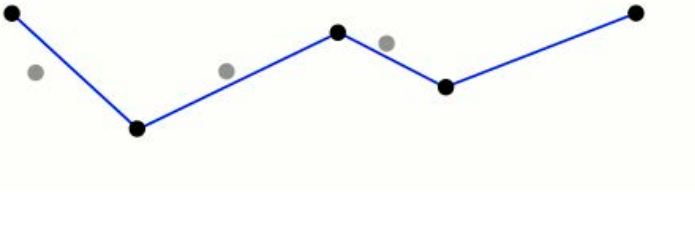


Si la distancia del peor punto al segmento es mayor o igual que ε, entonces se repite el proceso anterior de forma recursiva considerando:

1. El contorno de puntos entre el primer y el peor punto. ✓
2. El contorno de puntos entre el peor punto y el punto final de la curva.. ✓



Cuando se completa la recursión, la nueva curva puede ser generada a partir de los puntos que han permanecido tras haber aplicado el algoritmo.



Ejemplos en Python

In [1]:
```
import matplotlib.pyplot as plt
import numpy as np
```

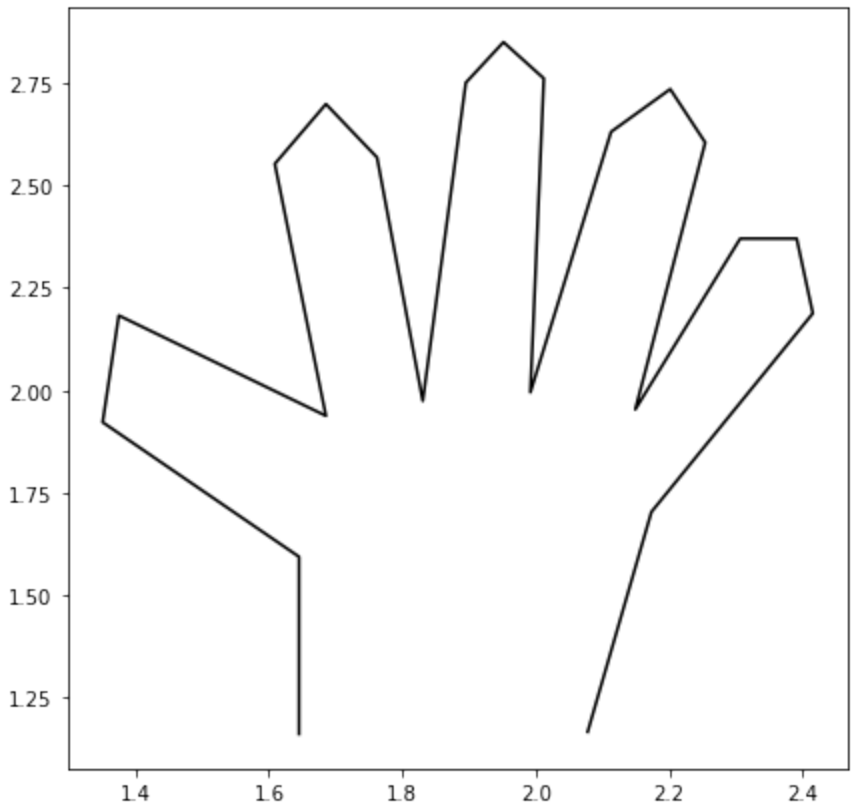
In [2]:
```
from skimage.measure import find_contours, approximate_polygon, subdivide_polygo
```

In [3]:
```
hand = np.array([[1.64516129, 1.16145833],
                 [1.64516129, 1.59375],
                 [1.35080645, 1.921875],
                 [1.375, 2.18229167],
```

```
                    [1.68548387, 1.9375],
                    [1.60887097, 2.55208333],
                    [1.68548387, 2.69791667],
                    [1.76209677, 2.56770833],
                    [1.83064516, 1.97395833],
                    [1.89516129, 2.75],
                    [1.9516129, 2.84895833],
                    [2.01209677, 2.76041667],
                    [1.99193548, 1.99479167],
                    [2.11290323, 2.63020833],
                    [2.2016129, 2.734375],
                    [2.25403226, 2.60416667],
                    [2.14919355, 1.953125],
                    [2.30645161, 2.36979167],
                    [2.39112903, 2.36979167],
                    [2.41532258, 2.1875],
                    [2.1733871, 1.703125],
                    [2.07782258, 1.16666667]])
```

In [4]:

```
plt.figure(figsize=(7, 7))
plt.plot(hand[:, 0], hand[:, 1], c="black")
plt.show()
```

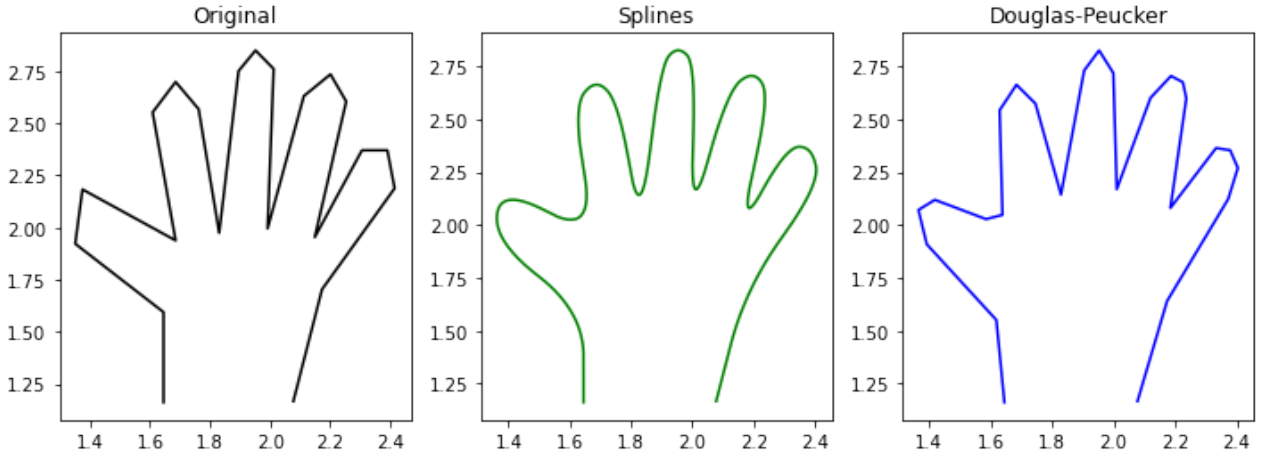


In [5]:

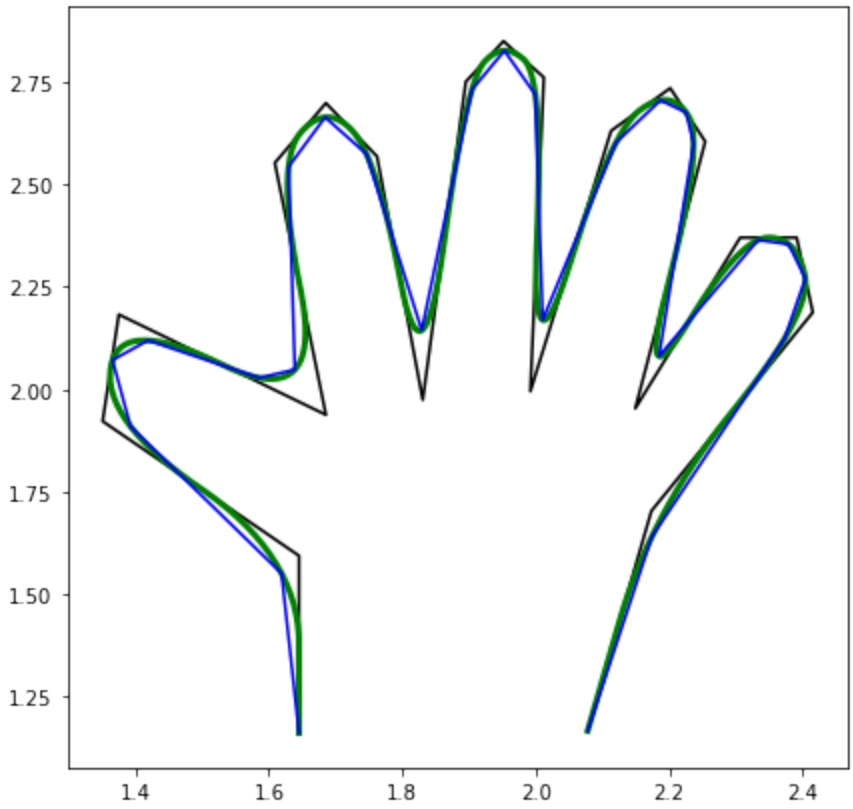
```
# Subdividir polígono usando Splines de segundo grado
new_hand = hand.copy()
for _ in range(5):
    new_hand = subdivide_polygon(new_hand, degree=2, preserve_ends=True)
```

In [6]:
```
# Polígono subdividido aproximado con algoritmo de Douglas-Peucker
appr_hand = approximate_polygon(new_hand, tolerance=0.02)
```

In [7]:
```
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(12, 4))
ax1.plot(hand[:, 0], hand[:, 1], c="black")
ax1.set_title('Original')
ax2.plot(new_hand[:, 0], new_hand[:, 1], c='green')
ax2.set_title("Splines")
ax3.plot(appr_hand[:, 0], appr_hand[:, 1], c='blue')
ax3.set_title("Douglas-Peucker")
plt.show()
```



In [8]:
```
fig, ax = plt.subplots(figsize=(7, 7))
ax.plot(hand[:, 0], hand[:, 1], c="black")
ax.plot(new_hand[:, 0], new_hand[:, 1], c='g', lw=3)
ax.plot(appr_hand[:, 0], appr_hand[:, 1], c='b')
plt.show()
```



Active Contour Model (Snakes)

- El "Modelo de Contorno Activo" es un método para ajustar *splines* a bordes en una imagen.
- Funciona minimizando una Energía que está definida en parte por la imagen y en parte por la forma del *spline*.
- El problema de encontrar el contorno se modela como un problema de minimización de energía.

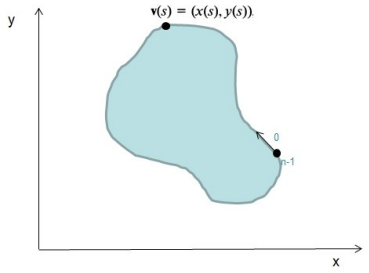
Framework for snakes

- Se inicializa cualquier curva cercana al borde de la imagen.
- La *snake* empieza a deformarse y se mueve hacia el borde deseado.
- Al final, la serpiente se envuelve alrededor del objeto.

Parametrización del Contorno

El contorno se define en el plano x, y de una imagen, como una curva paramétrica:

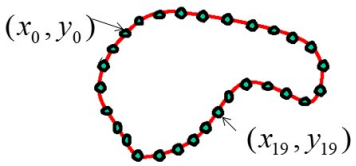
$$\mathbf{v}(s) = (x(s), y(s))$$



Representación

Es decir, es una representación discreta del contorno, que consiste en una lista de coordenadas en dos dimensiones ("vértices").

$$\mathbf{v}_i = (x_i, y_i), i = 0, \ldots, n-1$$

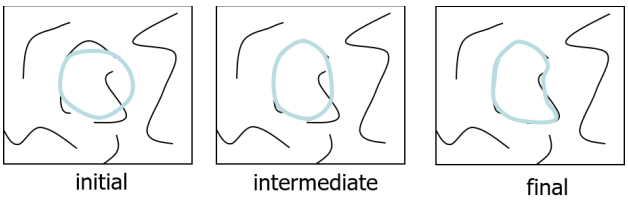


En cada iteración, tendremos la opción de mover cada vértice a otra ubicación cercana ("estado").

Ajuste de contornos deformables

¿Cómo debemos ajustar el contorno actual para formar el nuevo contorno en cada iteración?

- Defina una función de costo (función de "energía") que diga qué tan buena es una configuración candidata.
- Busque la siguiente configuración que minimice esa función de costo.



Función de Energía

El contorno de la imagen posee energías que de alguna forma ejercen cierta fuerza de acción sobre él. La Energía E_{snake} se define como la suma de la energía interna y la externa de la imagen:

$$E_{snake} = E_{interna} + E_{externa}$$

- Energía interna: Fomenta preferencias de forma previas. Por ejemplo, suavidad, elasticidad, forma particular conocida.
- Energía externa (energía de “imagen”): Anima al contorno a encajar en lugares donde existen estructuras de imagen, por ejemplo, bordes.

Un buen ajuste entre el contorno deformable actual y la forma del objetivo en la imagen producirá una energía baja.

Energía Interna

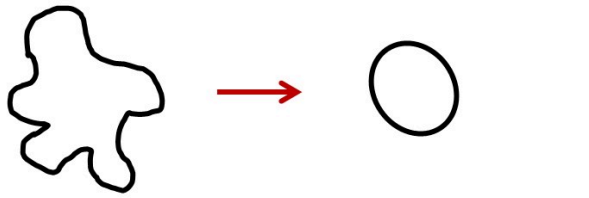
- Es la suma entre "elasticidad" y "rigidez".
- Elasticidad permite que la energía se pueda contraer.



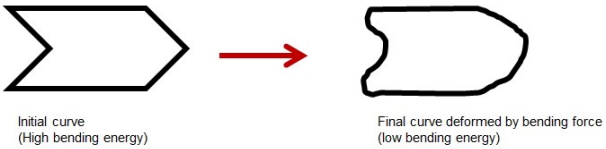
- Rigidez permite lograr suavidad en la curva.



Elastic (Elasticidad)



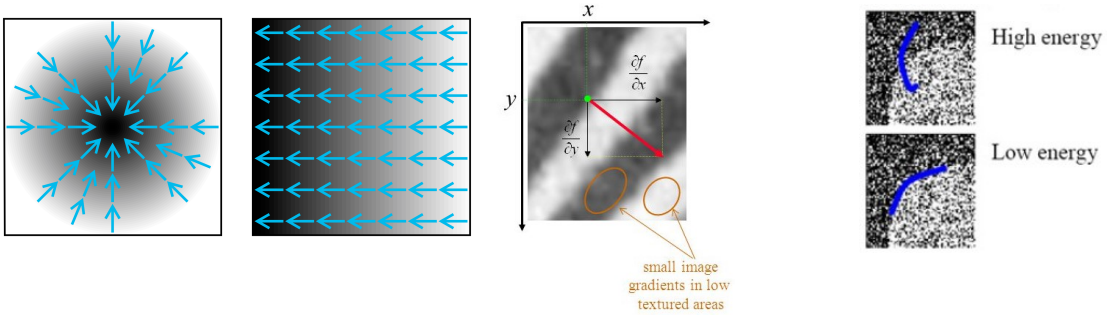
Bending or Stiffness (Rigidez)



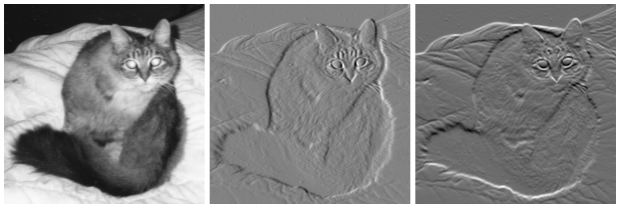
Energía Externa

- Mide qué tan bien la curva coincide con los datos de la imagen.

- “Atraer” la curva hacia diferentes características de la imagen como bordes, líneas, degradado de textura, etc.
- Piense en la energía externa de la imagen como un tirón gravitacional hacia áreas de alto contraste (donde el valor del gradiente es grande).



- Imágenes del Gradiente $G_x(x, y)$ y $G_y(x, y)$:



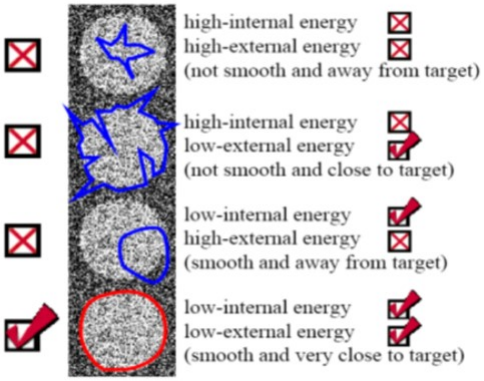
- Energía externa en un punto de la curva:

$$E_{externa}(v) = -(|G_x(v)|^2 + |G_y(v)|^2)$$

- Energía externa para la curva completa:

$$E_{externa} = -\sum_{i=0}^{n-1}(|G_x(x_i, y_i)|^2 + |G_y(x_i, y_i)|^2)$$

Energía del Contorno - Posibles casos

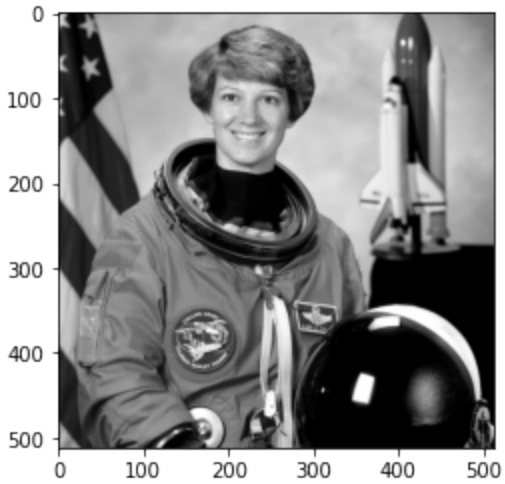


Ejemplo Active Contour Model (Snakes) en Python

In [9]:
```
from skimage import data
from skimage.color import rgb2gray
from skimage.filters import gaussian
from skimage.segmentation import active_contour
```

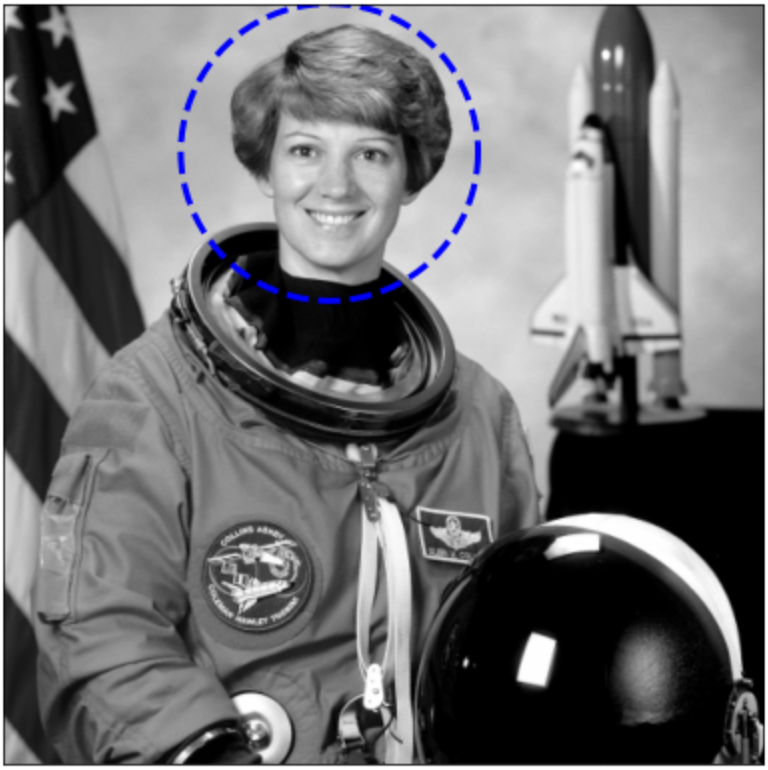
In [10]:
```
astronaut = data.astronaut()
astronaut = rgb2gray(astronaut)
```

In [11]:
```
plt.imshow(astronaut, cmap=plt.cm.gray)
plt.show()
```



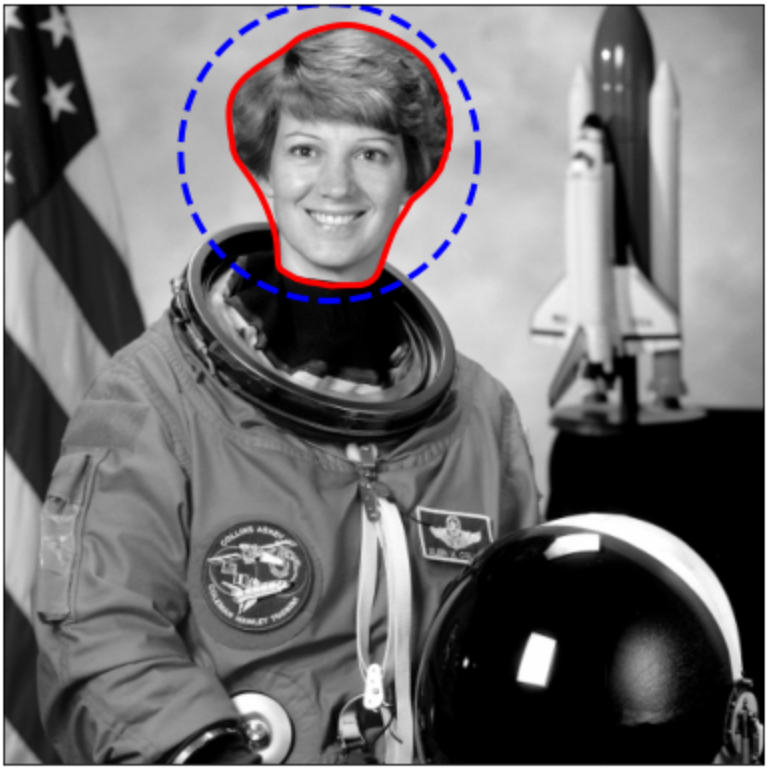
In [12]:
```
s = np.linspace(0, 2*np.pi, 400)
r = 100 + 100*np.sin(s)
c = 220 + 100*np.cos(s)
region = np.array([r, c]).T
```

In [13]:
```
fig, ax = plt.subplots(figsize=(7, 7))
ax.imshow(astronaut, cmap=plt.cm.gray)
ax.plot(region[:, 1], region[:, 0], '--b', lw=3)
ax.set_xticks([]), ax.set_yticks([])
ax.axis([0, astronaut.shape[1], astronaut.shape[0], 0])
plt.show()
```



In [14]:
```
def plot_snakes(img, region, sigma, alpha, beta, gamma):
    snake = active_contour(gaussian(img, sigma, preserve_range=False), region, a
    fig, ax = plt.subplots(figsize=(7, 7))
    ax.imshow(img, cmap=plt.cm.gray)
    ax.plot(region[:, 1], region[:, 0], '--b', lw=3)
    ax.plot(snake[:, 1], snake[:, 0], '-r', lw=3)
    ax.set_xticks([]), ax.set_yticks([])
    ax.axis([0, img.shape[1], img.shape[0], 0])
    plt.show()
```

In [15]:
```
plot_snakes(astronaut, region, sigma=3, alpha=0.015, beta=10, gamma=0.001)
```



Morphological Snakes

Morphological Snakes son métodos similares a los *Active Contours*, pero usan operadores morfológicos (como dilatación o erosión) sobre la imagen binaria.

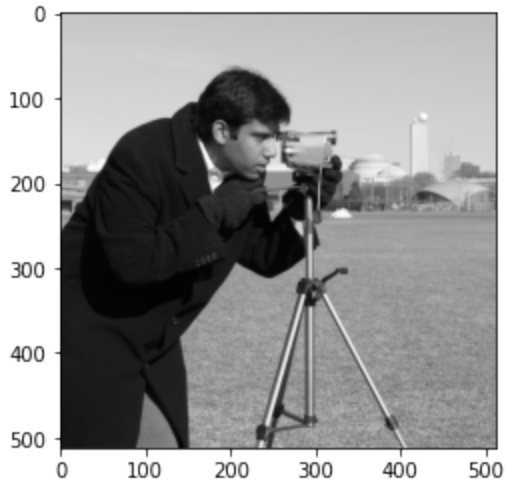
Chan-Vese

Active Contour Model implementado con operadores morfológicos mediante la evolución de un conjunto de niveles. Se puede utilizar para objetos sin bordes claramente definidos. Se requiere que el interior del objeto se vea diferente en promedio que el exterior (es decir, el área interior del objeto debe ser más oscura o más clara que el área exterior en promedio).

In [16]:
```
from skimage import data, img_as_float
from skimage.segmentation import morphological_chan_vese
```

In [17]:
```
camera = img_as_float(data.camera())
```

In [18]:
```
plt.imshow(camera, cmap=plt.cm.gray)
plt.show()
```



In [19]:

```
def plot_chan_vese(img, num_iter, smoothing):
    ls = morphological_chan_vese(img, num_iter=num_iter, smoothing=smoothing)
    fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(16, 8), sharex=True, sharey
    ax1.imshow(ls, cmap=plt.cm.gray)
    ax1.set_title('Chan-Vese')
    ax1.axis('off')
    ax2.imshow(img, cmap="gray")
    ax2.contour(ls, colors='r')
    ax2.axis('off')
    ax2.set_title('Contorno')
```

In [20]:

```
plot_chan_vese(camera, num_iter=35, smoothing=3)
```



